ChatGPT said:

Let's handle `fclose()` errors now. It's a small change, and it reinforces **resource management** in C.

Assignment 003B — Build our first WAV header

We're going to create a valid WAV file containing one second of silence. After that, we'll replace the silence with samples from your oscillator.

This is our first encounter with a real binary file specification.

Step 1 — Finish error handling

In your existing `write_test_file()` function, check the return value of `fclose()`.

Here's the contract:

```c
int fclose(FILE *stream);
```

- Returns `0` on success.
- Returns `EOF` on failure (`EOF` is defined in `<stdio.h>`).

Use `fprintf(stderr, ...)` to report a failure and return a nonzero error code.

Once that's working, move on to the WAV writer.

Step 2 — Understand the WAV file layout

A basic PCM WAV file has three major sections:

RIFF header — 12 bytes

Identifies the file as a RIFF/WAVE file

fmt chunk — 24 bytes

Describes the audio format: PCM, channels, sample rate, bit depth

data chunk header — 8 bytes

Identifies the audio data and its size

PCM samples — 96,000 bytes

The actual audio, initially silence

Together, the first three sections form our 44-byte WAV header.

Step 3 — Here is the binary specification

Every field must appear in exactly this order.

Offset	Size	Field	Value
0	4	ChunkID	`"RIFF"`
4	4	ChunkSize	`96036`
8	4	Format	`"WAVE"`
12	4	Subchunk1ID	`"fmt "`
16	4	Subchunk1Size	`16`
20	2	AudioFormat	`1` (PCM)
22	2	NumChannels	`1`
24	4	SampleRate	`48000`
28	4	ByteRate	`96000`
32	2	BlockAlign	`2`
34	2	BitsPerSample	`16`
36	4	Subchunk2ID	`"data"`
40	4	Subchunk2Size	`96000`

A few formulas explain those numbers:

$$\text{ByteRate} = \text{SampleRate} \times \text{Channels} \times \frac{\text{BitsPerSample}}{8}$$

$$\text{BlockAlign} = \text{Channels} \times \frac{\text{BitsPerSample}}{8}$$

$$\text{ChunkSize} = 36 + \text{AudioDataBytes}$$

All numerical fields in this WAV format use **little-endian byte order**, which you've already encountered while exploring memory in GDB.

Step 4 — Your programming assignment

Create a new function in `file_io.c`:

```c
int write_silent_wav(const char *filename);
```

Your function should:

1. Open the file in binary mode.
2. Write the 44-byte WAV header according to the table.
3. Write 48,000 samples of silence as 16-bit integers.
4. Close the file, checking for errors.
5. Return `0` for success or nonzero for failure.

You can use `<stdint.h>` for `uint16_t`, `uint32_t`, and `int16_t`.

For writing, continue using `fwrite()`. You already know how it works.

One hint for the header:

```c
uint32_t sample_rate = 48000;
fwrite(&sample_rate, sizeof(sample_rate), 1, file);
```

That writes the four bytes of the integer in their native memory order. On your x86-64 WSL environment, that is little-endian, as WAV requires. Later we'll discuss making this portable to other architectures.

For silence, every `int16_t` sample should be zero.

Step 5 — Verify your results

Compile and run your program, then inspect the file:

```bash
```

```
xxd -g 1 -l 44 silence.wav
```

You should see `RIFF`, `WAVE`, `fmt `, and `data` in the ASCII column.

Check the size:

Bash

```
wc -c silence.wav
```

Expected:

96,044 bytes

That's 44 bytes of header plus 96,000 bytes of audio.

You can also ask Linux to identify the format:

Bash

```
file silence.wav
```

One design concept to watch for

Serialization: We're taking meaningful values in our program and converting them into a standardized sequence of bytes that another program can interpret.

And one piece of advice: don't build a giant 44-byte array by hand unless you particularly want to. Consider writing the fields individually with `fwrite()`.

Your stopping point: Get the silent WAV file created and recognized by the `file` command.

Then we'll use your existing sine-wave generator to replace the silence, and you'll hear the first audio produced by `subosc`.